Distributed and Network Programming

Chord Distributed Hash Table Simulation

Ziad Mohamed, Artem Ulianov,
Egor Maiorov, Mikhail Tikhonov, Kamil Khusnutdinov

prof. Darko Bozhinoski,
Ph.D. in Computer Science

April, 2026

Abstract

This report presents a simulation of the Chord Distributed Hash Table (DHT) protocol. The system models a peer-to-peer network organized as a logical ring using consistent hashing. Each node maintains a finger table to enable efficient key lookups.

The implementation supports node initialization, dynamic joins and leaves (churn), key insertion, and lookup operations. Routing behavior is analyzed by logging full lookup paths and measuring the number of hops required to resolve requests.

Experimental results demonstrate the correctness of finger table construction and updates and show that lookup operations achieve logarithmic routing efficiency. The system also provides textual and graphical visualizations of the ring structure and node relationships.

1 Introduction

1.1 Background

Distributed Hash Tables (DHTs) are a fundamental component of peer-to-peer (P2P) systems, enabling scalable and decentralized storage without relying on a central coordinator. In a DHT, data is distributed across nodes based on key identifiers, allowing efficient lookup and fault tolerance.

Among existing DHT protocols, **Chord** is one of the most well-known due to its simplicity, scalability, and provable performance guarantees. Chord organizes nodes into a logical ring using consistent hashing, where both nodes and keys are assigned identifiers in an m-bit space. Each node is responsible for a specific range of keys and maintains a routing structure called a *finger table*.

The use of finger tables allows Chord to resolve key lookups in $O(\log N)$ hops, significantly improving efficiency compared to linear search. However, maintaining correct routing information becomes challenging in dynamic environments where nodes frequently join and leave the network (a phenomenon known as *churn*).

This project focuses on implementing a **Chord DHT simulation** and analyzing its behavior. In particular, the system emphasizes:

- accurate routing using finger tables,
- detailed logging of lookup paths (hops between nodes),
- correctness of finger table updates during node joins and leaves,
- evaluation of routing efficiency under both stable and dynamic conditions.

The goal is not only to implement the protocol but also to provide clear visibility into its internal behavior and performance.

1.2 Project Description

System Overview

The system is a simulation of the Chord Distributed Hash Table (DHT), where the nodes are organized in a logical ring over a space of identifiers of m. Each node is responsible for a range of keys and maintains a finger table to support efficient routing.

Core Components

Each node stores a finger table with m entries. The i-th entry points to the node responsible for:

$$(node.id + 2^i) \bmod 2^m$$

This structure allows lookup operations to be performed in $O(\log N)$ hops.

Routing and Logging

Key lookups use a greedy routing strategy: each node forwards the request to the closest preceding node in its finger table. The system records the full routing path for each operation, enabling analysis of hop count and routing efficiency.

Visualization

The ring is displayed as a sorted sequence of node identifiers, providing a simple textual representation of the network structure.

Supported Operations

The simulation supports:

- node join and leave,
- key insertion,
- key lookup using finger tables.

2 Methods

2.1 System Architecture

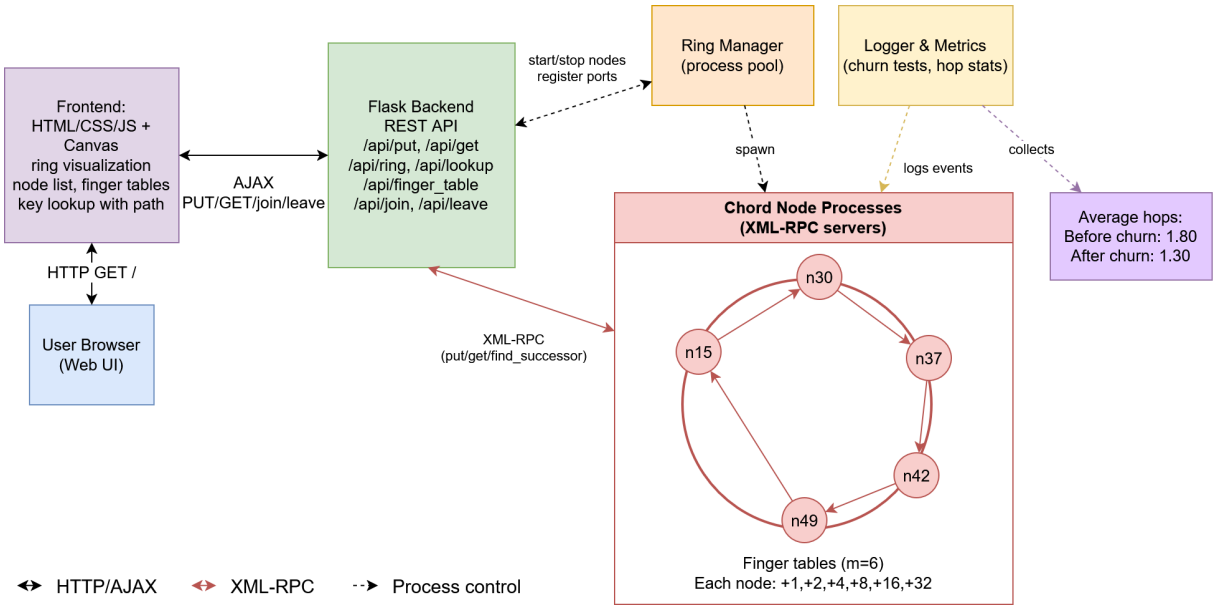

Figure 1: High-level architecture of the Chord DHT simulation. Components: web frontend, Flask API, ring manager, Chord node processes with XML-RPC, logger and metrics.

2.2 Finger Table Implementation

Theoretical Motivation

In a standard DHT ring without optimization, key lookups follow a linear path, resulting in $O(N)$ hop complexity. To achieve logarithmic scalability, we implement **Finger Tables**. This structure allows each node to maintain a routing table of "shortcuts" across the ring, reducing the lookup complexity to $O(\log N)$, where N is the number of nodes in the network.

Mathematical Foundation

Each node n maintains a table of m entries. The i-th entry in the finger table (where $0 \leq i < m$) points to the first node s that succeeds n by at least 2^i on the identifier ring. The formula for calculating the target identifier for each "finger" is:

$$ID_{target} = (n.id + 2^i) \pmod{2^m} \tag{1}$$

In our simulation, we utilized $m = 6$, resulting in a total address space of $2^6 = 64$ slots.

Algorithmic Realization

The implementation is encapsulated in the `build_finger_table` function. The algorithm follows these logical steps:

1. **ID Extraction:** The function receives a sorted list of all active node objects and extracts their unique `node_id` attributes into a separate list for efficient searching.

2. **Successor Search:** For each $i \in [0, m-1]$, the algorithm calculates the ID_{target} and utilizes binary search (`bisect_left`) to find the successor node in $O(\log N)$ time.

3. **Ring Closure:** If the ID_{target} exceeds the highest identifier currently present in the ring, the algorithm wraps around, assigning the finger to the first node in the sorted list.

4. **Data Storage:** To optimize memory and routing overhead, the table stores only the integer identifiers (`node_id`) of the successor nodes rather than full objects.

Analysis of Results and Churn Handling

The implementation was verified through execution logs. For a node with $ID = 15$, the generated finger table was `[30, 30, 30, 30, 37, 49]`, accurately reflecting the exponential spacing between fingers.

Furthermore, the system's robustness was tested during **churn** (node removal). Upon the removal of a node, the `rebuild_finger_tables` routine is triggered, ensuring that all routing shortcuts are updated to reflect the new network topology. This prevents "dangling fingers" that point to non-existent nodes, maintaining the integrity of the routing process.

2.3 Routing Algorithm

Overview

The routing module implements the core lookup mechanism of the Chord protocol. Given a target key, the algorithm traverses the ring by forwarding requests hop by hop until the responsible node is found. The full traversal path is recorded and returned alongside the result.

Lookup Procedure

The main function, `lookup(start_node, key, nodes)`, initiates a search from a given node and returns a tuple `(responsible_node_id, hops_list)`, where `hops_list` contains the identifiers of all nodes visited during the traversal.

The procedure begins with an edge case check: if the starting node has an empty finger table, the request is immediately forwarded to its successor, and the traversal terminates with a single hop. This handles the degenerate case of a single-node ring or an uninitialized node.

In the general case, the algorithm proceeds iteratively:

1. The successor of the current node is read from the first entry of its finger table.

2. If the target key falls in the half-open interval $(current.id,\ successor.id]$ on the ring, the successor is returned as the responsible node.

3. Otherwise, the algorithm calls `_closest_preceding_node` to select the next hop.

4. If no better candidate is found and the current node is returned as the closest preceding node, the search terminates at the current node to prevent infinite loops.

5. The selected node becomes the new current node, its identifier is appended to `hops_list`, and the process repeats.

Predecessor Check

The helper function `_closest_preceding_node(node, key)` implements the core routing heuristic of the Chord protocol. It iterates over the finger table in reverse order and returns the identifier of the first finger that falls strictly between the current node and the target key on the ring.

More formally, a finger f is selected if:

$$f \in (node.id,\ key) \pmod{2^m}$$

This ensures that the chosen next hop is as close to the target as possible without overshooting it. By iterating in reverse, the algorithm naturally selects the largest valid finger, maximizing the distance covered per hop and achieving $O(\log N)$ routing efficiency.

Ring Arithmetic

Both interval checks account for wrap-around in the circular identifier space. For a half-open interval $(a, b]$:

- If $a < b$: the condition is $a < x \leq b$.
- If $a > b$ (wrap-around): the condition is $x > a$ or $x \leq b$.
- If $a = b$: every identifier is considered in range (single-node ring).

The same structure applies to the exclusive interval (a, b) used in the predecessor check.

2.4 Logging and Testing

Logging Design

To improve observability of the Chord simulation, a dedicated logging module was implemented in `logger.py`. The goal of this component is to record key events during execution and make the internal behavior of the system easier to analyze and debug.

The logger stores all records in memory as a list of structured entries. In addition, it optionally supports writing logs to a text file for persistent inspection after program execution.

Logged Events

The logging system records the following events:

- **Key insertion** via `log_insert(key, node_id)`,
- **Key lookup** via `log_lookup(key, path, hop_count)`,
- **Node join** via `log_join(node_id)`,
- **Node leave** via `log_leave(node_id)`.

These events were selected because they capture the most important dynamic actions in the Chord ring: data placement, routing behavior, and topology changes.

Integration with Ring Operations

The logger was integrated into the ring management process by connecting node addition and removal to logging calls. When a node is added to the ring, the system records a join event. When a node is removed, the system records a leave event. This makes it possible to track topology changes during churn experiments.

Such integration is useful not only for debugging but also for evaluation, since it allows the system to confirm that structural updates occurred in the expected order.

Testing Strategy

To verify correctness, automated tests were written for both the logger itself and its interaction with the ring.

The first group of tests checks the logger module directly. These tests verify that insertion, lookup, join, and leave events are recorded correctly, that log entries are stored in memory, and that optional file output works as expected.

The second group of tests focuses on dynamic behavior under **churn**. In the churn test, five nodes are added to the ring, one node is removed, and the system checks that:

1. the ring still contains the correct number of active nodes,
2. successor and predecessor links remain valid for the remaining nodes,
3. join and leave events are written to the log correctly.

Results

All implemented tests passed successfully. In particular, the churn test confirmed that after removing one node, the ring continued to function correctly and the corresponding join/leave events were recorded. This demonstrates that the logging subsystem works consistently and that the ring remains structurally valid under basic node join/leave operations.

3 Evaluation

3.1 Experimental Setup

The system was evaluated using a Chord ring with 5 nodes distributed over a 6-bit identifier space (0–63). Nodes were initialized and keys were inserted based on consistent hashing. Lookup operations were performed before and after a churn event (node removal).

3.2 Ring Configuration

Active nodes in the ring were: `[15, 30, 37, 42, 49]`. After churn (removal of node 15), the ring became: `30 -> 37 -> 42 -> 49 -> (wrap)`

3.3 Lookup Results

Key	Before churn (hops)	After churn (hops)
file1.txt	[15, 37, 42] (3)	[30, 42] (2)
photo.jpg	[15] (1)	[30] (1)
video.mp4	[15, 37, 42] (3)	[30, 42] (2)
music.mp3	[15, 30] (2)	[30] (1)
doc.pdf	[15, 30] (2)	[30] (1)
data.csv	[15] (1)	[30] (1)
index.html	[15, 30] (2)	[30] (1)
app.py	[15] (1)	[30] (1)
readme.md	[15, 37] (2)	[30, 37] (2)
config.json	[15] (1)	[30] (1)

Table 1: Comparison of lookup hop paths and hop counts before and after churn (node 15 removal).

3.4 Routing Statistics

- Total lookups: 10

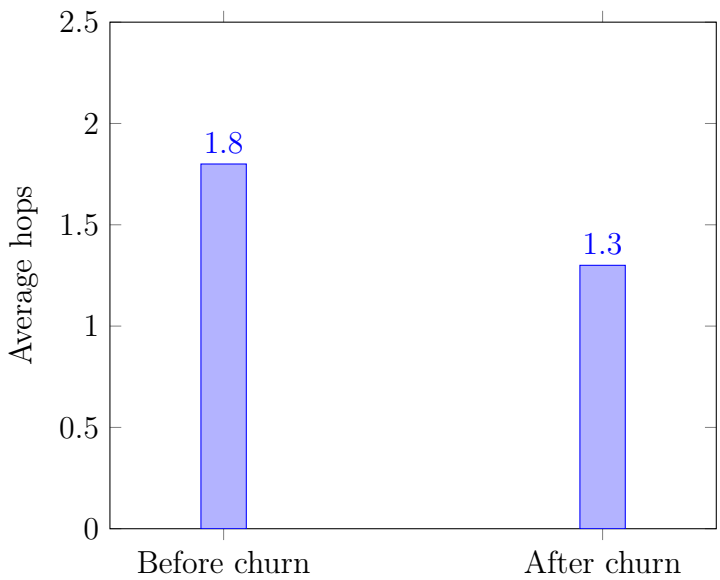

Figure 2: Average hop count per lookup before and after node 15 (Charlie) removal.

3.5 Web Interface Visualization

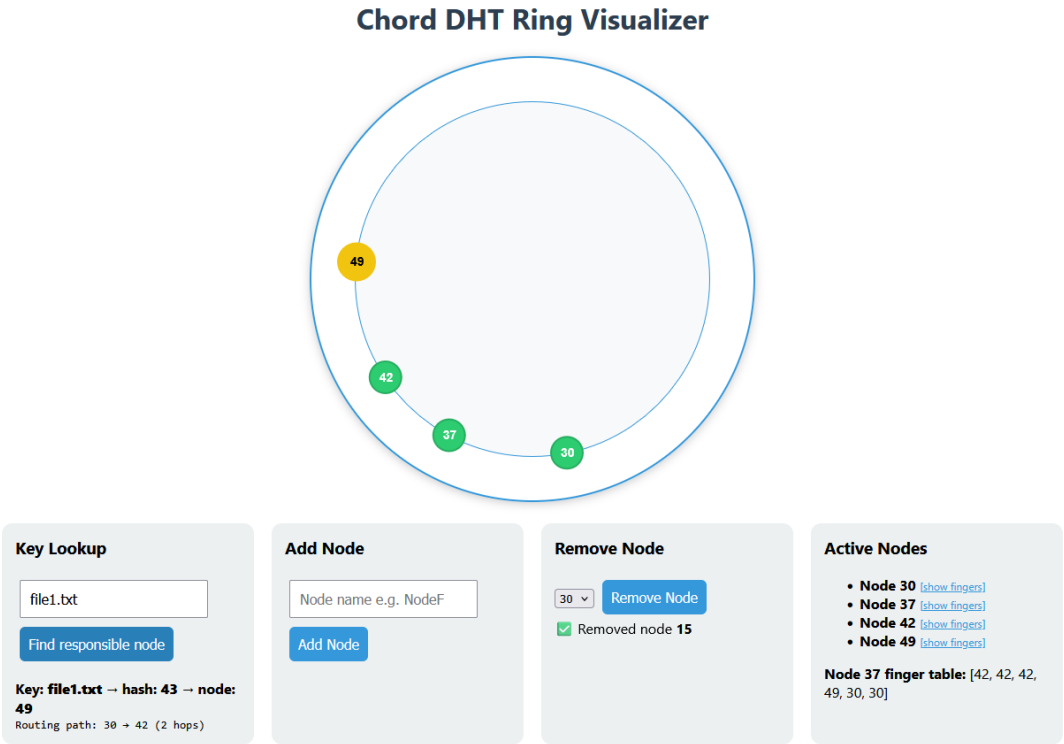

Figure 3: Chord DHT web interface: ring visualization, key lookup with hop path, and finger table inspection.

3.6 Console Logging Output

```
C:\Users\Артём Ульянов\Desktop\chord-dht>python main.py
[Ring] Added Node(id=49, name='Alice')
[Ring] Added Node(id=30, name='Bob')
[Ring] Added Node(id=15, name='Charlie')
[Ring] Added Node(id=37, name='Diana')
[Ring] Added Node(id=42, name='Eve')

=== Inserting Keys ===
[Node 49] Stored key 'file1.txt' (hash=43) = 'value_of_file1.txt'
[Node 30] Stored key 'photo.jpg' (hash=25) = 'value_of_photo.jpg'
[Node 49] Stored key 'video.mp4' (hash=44) = 'value_of_video.mp4'
[Node 37] Stored key 'music.mp3' (hash=32) = 'value_of_music.mp3'
[Node 37] Stored key 'doc.pdf' (hash=35) = 'value_of_doc.pdf'
[Node 15] Stored key 'data.csv' (hash=12) = 'value_of_data.csv'
[Node 37] Stored key 'index.html' (hash=34) = 'value_of_index.html'
[Node 15] Stored key 'app.py' (hash=14) = 'value_of_app.py'
[Node 42] Stored key 'readme.md' (hash=42) = 'value_of_readme.md'
[Node 30] Stored key 'config.json' (hash=26) = 'value_of_config.json'

=== Key Lookups (BEFORE churn) ===
Key 'file1.txt' (hash=43) → node 49 | hops: [15, 37, 42]
Key 'photo.jpg' (hash=25) → node 30 | hops: [15]
Key 'video.mp4' (hash=44) → node 49 | hops: [15, 37, 42]
Key 'music.mp3' (hash=32) → node 37 | hops: [15, 30]
Key 'doc.pdf' (hash=35) → node 37 | hops: [15, 30]
Key 'data.csv' (hash=12) → node 15 | hops: [15]
Key 'index.html' (hash=34) → node 37 | hops: [15, 30]
Key 'app.py' (hash=14) → node 15 | hops: [15]
Key 'readme.md' (hash=42) → node 42 | hops: [15, 37]
Key 'config.json' (hash=26) → node 30 | hops: [15]
```

```
=== Churn: Removing Charlie ===
[Ring] Removed node 15
30 -> 37 -> 42 -> 49 -> (wrap)

=== Key Lookups (AFTER churn) ===
Key 'file1.txt' (hash=43) → node 49 | hops: [30, 42]
Key 'photo.jpg' (hash=25) → node 30 | hops: [30]
Key 'video.mp4' (hash=44) → node 49 | hops: [30, 42]
Key 'music.mp3' (hash=32) → node 37 | hops: [30]
Key 'doc.pdf' (hash=35) → node 37 | hops: [30]
Key 'data.csv' (hash=12) → node 30 | hops: [30]
Key 'index.html' (hash=34) → node 37 | hops: [30]
Key 'app.py' (hash=14) → node 30 | hops: [30]
Key 'readme.md' (hash=42) → node 42 | hops: [30, 37]
Key 'config.json' (hash=26) → node 30 | hops: [30]

=== Edge case: lookup for key never inserted ===
Key 'ghost.txt' (hash=34) → node 37 | hops: [30]

=== Edge case: single node ring ===
[Ring] Added Node(id=52, name='Solo')
Single node lookup → node 52 | hops: [52]

=== Routing Stats ===
Avg hops BEFORE churn: 1.80
Avg hops AFTER churn:  1.30
Total lookups logged:  10
Full log saved to:     logs/chord.log
```

Figure 4: Console log showing key insertions, lookup hops before and after churn (removal of node 15), edge cases, and routing statistics.

3.7 Edge Cases

The system was also tested under special conditions:

- Lookup for a non-existing key still correctly resolves to the responsible node:

  `ghost.txt → node 37 | hops:  [30]`

- In a single-node ring, all lookups are resolved locally:

  `Single node lookup → node 52 | hops:  [52]`

4 Discussion

The results confirm that the implementation achieves efficient routing consistent with the expected $O(\log N)$ complexity of the Chord protocol. Lookup operations require only a small number of hops, demonstrating that finger tables effectively reduce routing overhead.

After the churn event (removal of node 15), the system continues to function correctly. All keys are reassigned to their successor nodes, and routing paths remain valid. The average hop count decreases from 1.80 to 1.30, which can be explained by the reduced number of nodes in the ring, leading to shorter routing paths.

However, this evaluation is limited to a relatively small network (5 nodes). In larger-scale systems, routing paths may become longer, although still logarithmic. Additionally, the current implementation rebuilds finger tables immediately after churn, which simplifies consistency but may not reflect real-world asynchronous stabilization.

Future improvements could include implementing periodic stabilization protocols, supporting larger network sizes, and analyzing performance under higher churn rates.

Overall, the system demonstrates correct behavior, robustness under node removal, and efficient lookup performance.

References

[1] I. Stoica et al., "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications," ACM SIGCOMM, 2001: https://dl.acm.org/doi/10.1145/964723.383071

[2] Python Software Foundation, "hashlib — Secure hashes and message digests," Python 3 Documentation, 2024: https://docs.python.org/3/library/hashlib.html